

UML Modeling Diagrams

Your notes contain a very detailed use-case documentation template (iteration, priority, channel-to-actor, frequency of use, etc.) — per your call, this draft covers the diagramming concepts and when to use each diagram, but skips that documentation-field checklist as too granular for interview purposes.

1. Data Flow Diagram (DFD)

What It Is

A **Data Flow Diagram** is a graphical representation of how data flows through a system — how it enters, how it's processed, where it's stored, and how it leaves — **without** showing implementation or program logic. DFD focuses on **WHAT** happens to data, not **HOW** it's implemented.

Components

Symbol	Meaning	Drawn As	Examples
External Entity	A person, organization, or system outside the system boundary; sends/receives data but doesn't process it	Rectangle	Customer, Payment Gateway, Admin
Process	An operation transforming input data into output data; must have ≥ 1 input and ≥ 1 output	Circle / rounded rectangle	Verify Login, Process Payment
Data Flow	Movement of data between entities, processes, and stores; must be named	Arrow	Login Details, Account Balance
Data Store	A place where data is stored, read from, or written to	Open-ended rectangle	Account Database, User Credentials

Levels of DFD

Level -1 (Context Diagram)	→ the ENTIRE system as one process, with external entities + data flows only
Level 0	→ system expanded into major sub-processes + data stores
Level 1	→ one Level-0 process decomposed further e.g. "2.0 Transfer Funds" → 2.1 Validate Account, 2.2 Check Balance, 2.3 Transfer Amount, 2.4 Update Records
Level 2+	→ used only when a process is still too complex; each level adds detail

Rules of DFD (frequently tested)

1. Every process must have at least one input and one output.
2. Data **cannot** flow directly: entity → entity, store → store, or entity → data store (it must always pass through a process).
3. Every data flow must be named.
4. DFDs must be **balanced** — inputs and outputs at one level must match the corresponding process's inputs/outputs at the next level down.
5. A DFD does **not** show control flow, loops, or decisions — that's what Activity/State diagrams are for.

Advantages / Limitations

Advantages	Limitations
Easy to understand, improves communication	Doesn't show timing or sequence
Helps in requirement analysis	Not suitable for real-time systems
Independent of programming language	Becomes unwieldy for very large systems
Identifies missing or redundant processes	Doesn't represent decision logic clearly

2. Use Case Diagram

What It Is

A UML diagram showing how **actors** (users or external systems) interact with a system — it describes **WHAT** the system does from the user's point of view, not how it's internally implemented.

Components

Symbol	Meaning	Drawn As
Actor	A user or external system interacting with the system, always outside the boundary	Stick figure
Use Case	A function/service the system provides	Oval
System Boundary	Defines the system's scope — everything inside belongs to it	Rectangle
Association	Shows an actor participates in a use case	Simple line

Relationships Between Use Cases

Relationship	Meaning	Example
<<include>>	One use case <i>always</i> uses another — used to avoid repeating shared steps	"Transfer Funds" includes "Authenticate User" → authentication is mandatory for a transfer
<<extend>>	One use case <i>optionally</i> extends another, only under specific conditions	"Apply for Loan" extends "Check Eligibility" → eligibility check only happens when a loan is actually applied for
Generalization	Shows inheritance between actors or use cases — a child inherits behavior from a parent	User → Admin (Admin is a specialized User)

Include vs. Extend, in one line: *include* is mandatory and unconditional ("this always happens as part of that"); *extend* is optional and conditional ("this happens sometimes, only if a condition holds").

Example: Online Banking System

- **Actors:** Account Holder, Payment Gateway
- **Use Cases:** Login, Change Password, Check Balance, Transfer Funds, Pay Bills, Request Statement, Request Cheque Book

Rules

1. Use cases must be clear and action-oriented (named as a verb phrase — "Transfer Funds," not "Funds").

2. Actors must stay outside the system boundary.
3. Avoid technical/implementation details — a use case diagram describes behavior, not internal logic.
4. It shows behavior, not data flow (that's the DFD's job).
5. One actor can participate in multiple use cases, and vice versa.

Advantages / Limitations

Advantages	Limitations
Easy to understand, user-friendly	Doesn't show internal logic
Helps in requirement gathering	No data-flow representation
Clearly defines system functionality/scope	Cannot show the <i>sequence</i> of actions
Useful for planning and testing	Not suitable for detailed design

A use case diagram only shows *that* "Book Train Ticket" is a thing a Customer can do — it deliberately says nothing about the steps, error conditions, or business rules involved. When that level of detail is needed, teams write a separate **formal use case document** (per use case) fleshing out its normal flow, exception flows, and preconditions in prose — but the diagram itself intentionally stays at the "what, not how" level.

3. Activity Diagram

What It Is

An Activity Diagram shows how work flows from start to end, step by step, including decisions and parallel actions — essentially **a flowchart, in UML form**. It models the behavior of a system, the workflow of a process, or the internal logic inside one specific use case, answering: "*in what order do actions happen?*"

It belongs to **Behavioral Modeling** (specifically, control-flow behavior) — unlike a Use Case diagram, which shows the user's external view, an Activity Diagram shows the **internal** flow of steps.

Components

Symbol	Meaning	Drawn As
Initial Node	Where the activity starts	Solid black circle
Activity / Action	A single step or operation	Rounded rectangle
Control Flow	Sequence between actions	Arrow
Decision Node	A branch point (if/else)	Diamond
Merge Node	Combines alternative paths back into one	Diamond
Fork Node	Splits flow into parallel activities	Thick bar
Join Node	Joins parallel flows back into one	Thick bar
Final Node	Marks the end of the activity	Circle with a dot inside

Example: Online Banking — Pay Bill

```
(start)
|
User selects "Pay Bill"
|
Enter bill details
|
Verify balance
|
<Sufficient balance?>
| Yes      | No
Process payment  Show error
|              |
Update account  (end)
|
(end)
```

4. Swimlane Diagram

A **Swimlane Diagram** is a type of Activity Diagram that additionally shows **who** does what in a process — it divides the activity diagram into lanes, where each lane represents a responsible entity (a person, department, system, or role). Instead of only showing *what* happens, it also shows *who is responsible* for each step.

Why "swimlane"? Like swimmers in a pool, each staying in their own lane — actions don't mix across lanes. Each lane = one actor, and every activity is placed inside the lane of whichever actor performs it. This is the natural diagram to reach for whenever a process involves a hand-off between multiple people/systems (e.g., "Customer submits form" → "Clerk reviews" → "Manager approves") and you specifically want to make those hand-offs visible.

5. State Diagram

A State Diagram models how a *single object* behaves over time, moving between distinct **states** in response to **events**.

Building Blocks

States — a condition of an object at a specific time. Two kinds:

- **Passive state** — described by attribute values (e.g., a player's position, magic points)
- **Active state** — described by behavior/activity (e.g., Moving, At Rest, Injured)

For a state diagram, the focus is on **active** states — e.g., for a `ControlPanel` class: `Reading` , `Comparing` , `Locked` , `Selecting` .

Events (triggers) — cause the object to move from one state to another. *Example:* `Key hit` , `Password entered` . Written directly on the transition arrow.

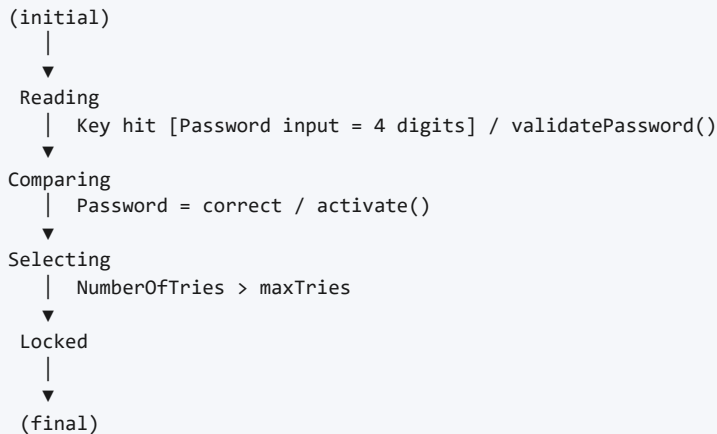
Guards — a boolean condition that must be true for a transition to actually fire. *Example:* `Password = incorrect & numberOfTries < maxTries` . Guards are optional, written in square brackets on the transition arrow: `[Password = incorrect & numberOfTries < maxTries]` .

Actions — an operation performed as a result of the transition. *Example:* `validatePassword()` . Written after a slash on the arrow: `/validatePassword()` .

Notation Summary

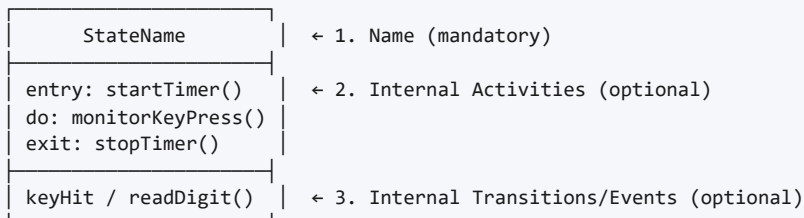
Symbol	Meaning
Rounded rectangle	Active state
Solid black circle	Initial state
Black circle with outer ring	Final state
Arrow	Transition
<code>Event [Guard] / Action</code>	Full label on a transition arrow

Worked Example: `ControlPanel`



The Three Compartments of a State

A UML state box can be split into three horizontal compartments:



1. **State Name** (mandatory) — what the object is "doing" or "experiencing," e.g., `Reading`, `Locked`.
2. **Internal Activities** (optional) — three kinds: `entry:` (runs once, when entering the state — e.g., `entry: startTimer()`), `do:` (runs continuously *while* in the state — e.g., `do: monitorKeyPress()`), `exit:` (runs once, when leaving the state — e.g., `exit: stopTimer()`).
3. **Internal Transitions/Events** (optional) — events handled *without leaving the state* (a self-transition), e.g., `keyHit / readDigit()`.

Why bother splitting the box into compartments at all? It separates three genuinely different things that would otherwise get muddled together in a single label: the state's *identity*, what it *does continuously while active* (independent of any transition), and how it *responds to specific events without actually leaving*. This is especially useful for modeling active states with genuinely ongoing behavior (like `Reading` continuously monitoring key presses), not just states that are pass-through waypoints between transitions.

When to Use Which Diagram

Diagram	Answers	Shows
DFD	Where does data come from, go to, and get stored?	Data movement through the whole system
Use Case	What can each type of user do?	System functionality from the outside, user's POV
Activity	In what order do the steps of one process happen?	Internal step-by-step workflow, including branches and parallelism
Swimlane	Who is responsible for each step of a process?	Same as Activity, plus ownership per step
State	How does one object's behavior change over time?	An object's lifecycle as a set of states and transitions

Interview Questions With Answers

Q1. Why can't a Data Flow Diagram show a data flow directly from one external entity to another, or directly from an entity to a data store?

Answer: A DFD's core rule is that data can only be transformed or routed by going through a **process** — external entities and data stores are, by definition, outside the system's active processing logic (an entity is a source/sink of data, and a data store is passive storage), so a direct flow between two entities, or from an entity straight into a store, would represent data moving through the system without anything actually happening to it, which isn't a meaningful concept in DFD's model. If, in reality, data genuinely needs to move from an entity into storage, that has to be represented as the entity providing data to a process (e.g., "Validate and Save Record"), which then writes to the data store — this forces the diagram to make explicit exactly which process is responsible for that data ending up in storage, rather than leaving it ambiguous.

Q2. What's the practical difference between `<<include>>` and `<<extend>>` in a Use Case Diagram, and why does mixing them up matter?

Answer: `<<include>>` models a mandatory, unconditional relationship — the including use case *always* triggers the included one as part of its normal flow, with no condition attached (e.g., "Transfer Funds" always includes "Authenticate User" — you cannot transfer funds without authentication happening). `<<extend>>` models an optional relationship that only applies under a specific condition — the extending use case only runs *sometimes*, when a particular circumstance holds (e.g., "Apply for Loan" extends "Check Eligibility" only in the sense that a loan application scenario invokes an eligibility check as an optional add-on step at a specific point, not as a mandatory prerequisite for every interaction with the base use case). Mixing them up matters because it changes what the diagram is

actually claiming is true about the system: mislabeling a mandatory step as `<<extend>>` implies to anyone reading the diagram that the step is optional/conditional, potentially leading to a design or test plan that doesn't treat that step as something that must always happen.

Q3. How does an Activity Diagram differ from a Use Case Diagram, given that both can describe "what a system does"?

Answer: A Use Case Diagram describes the system's functionality from the **outside**, from the user's perspective — it lists *what* the system can do (its use cases) and *who* can invoke each one (actors), but it deliberately says nothing about the steps, order, or internal logic involved in accomplishing any single use case. An Activity Diagram, by contrast, models the **internal** step-by-step workflow of a single process — often specifically the internal logic behind one particular use case — showing the actual sequence of actions, decision points, and even parallel activities involved. In short: the Use Case Diagram answers "what can happen," while the Activity Diagram (often drawn for one specific use case from that diagram) answers "and when it does happen, in what order do the individual steps occur."

Q4. What extra information does a Swimlane Diagram carry that a plain Activity Diagram doesn't, and when would you specifically choose to draw one?

Answer: A plain Activity Diagram shows the sequence of actions and decisions in a process, but it doesn't indicate *who* or *what* is responsible for performing each action. A Swimlane Diagram adds exactly that by dividing the diagram into lanes, one per responsible actor (a person, role, department, or system), and placing each activity inside the lane of whoever performs it. You'd specifically choose a swimlane diagram over a plain activity diagram whenever a process involves a **hand-off between multiple parties** — for example, a loan application that moves from Customer, to Clerk, to Manager, to Payment System — because in that situation, seeing *who* is responsible at each step is exactly the information a plain activity diagram would leave out, and is often the most important thing stakeholders in different departments actually want to see clearly.

Q5. In a State Diagram, what's the difference between a guard and an action, and can a single transition have both?

Answer: A guard is a boolean condition that must evaluate to true for a transition to actually fire at all — it's a gate-keeping check written in square brackets (e.g., `[numberOfTries < maxTries]`), and if the guard is false when the triggering event occurs, the transition simply doesn't happen. An action is an operation that's *performed as a result of* the transition actually firing — written after a slash (e.g., `/validatePassword()`) — it's a side effect, not a condition; the transition doesn't depend on whether the action succeeds or fails, since the action only runs once the transition (including its guard, if any) has already been determined to occur. Yes, a single transition can absolutely have both — the standard full label format is `Event [Guard] / Action`, meaning: "when this event occurs, if this guard is true, make this transition and then perform this action" — for example, `Key hit [Password input = 4 digits] / validatePassword()` from the ControlPanel example fires on the `Key hit` event, only if 4 digits have been entered, and then performs `validatePassword()` as a consequence.

Q6. Explain the difference between a state's `entry:`, `do:`, and `exit:` internal activities, using the `Reading` state example.

Answer: `entry:` specifies an action that runs exactly once, at the instant the object *enters* the state — in the `Reading` state example, `entry: displayPrompt()` means the prompt is displayed the moment the object transitions into `Reading`, regardless of how it got there. `do:` specifies an activity that runs *continuously*, for as long as the object remains in that state — `do: monitorKeyPress()` means the system keeps watching for key presses throughout the entire time the object stays in `Reading`, not just once. `exit:` specifies an action that runs exactly once, at the instant the object *leaves* the state, regardless of which transition caused it to leave — this is where cleanup logic typically goes (e.g., stopping a timer that was started on entry). The distinction matters because conflating them would make it ambiguous whether a piece of behavior happens once (on arrival, or on departure) or continuously throughout the state's duration — which is exactly the kind of ambiguity that leads to bugs like a timer being started multiple times or never being stopped.

Q7. Why does a DFD explicitly refuse to show decision logic or control flow, when that seems like useful information about a system?

Answer: A DFD's entire purpose is to show data movement — where data comes from, how it's transformed, where it's stored, and where it ultimately goes — independent of the specific logic or sequencing that governs when each transformation happens; this is a deliberate scoping decision, not an oversight. Mixing in control flow and decision logic would blur DFD's "what happens to data" focus with the "in what order, and under what conditions, do things happen" focus that Activity and State diagrams are specifically designed to handle — trying to cram both into one diagram type would make it simultaneously harder to read as a data-movement map and worse at representing control flow than a dedicated diagram. This is really the same "separation of concerns" principle from the Design Concepts topic, applied to choosing which UML diagram to use for which question, rather than trying to build one diagram that answers every possible question about a system at once.

Q8. Scenario: You're documenting an ATM system. A reviewer asks you to show (a) how cash withdrawal data flows between the ATM, the bank's database, and the card network, (b) what actions a Customer versus a Bank Admin can each perform, and (c) how a single ATM card-reader component's state changes as a card is inserted, read, retained, or ejected. Which UML/data-modeling diagram would you use for each, and why would using just one diagram for all three be a mistake?

Answer: For (a), a Data Flow Diagram is the right tool, since the question is specifically about data movement — how withdrawal-related data passes between the ATM, the database, and the card network — which is exactly DFD's domain, while it correctly avoids getting into card-reader internals or user-permission distinctions, which aren't data-flow questions at all. For (b), a Use Case Diagram is the right tool, since the question is about *what different actors can do* — Customer vs. Bank Admin capabilities — which is precisely what actors and use cases are for, without needing to show any internal step-by-step logic or state changes. For (c), a State Diagram is the right tool, since the question is specifically about how one component (the card reader) changes state over time in response to events (card

inserted, read completed, card retained, card ejected) — exactly State Diagram's domain. Using just one diagram type for all three would be a mistake because each diagram type is deliberately scoped to answer one specific kind of question well by *excluding* other kinds of information (DFD excludes control flow and actors; Use Case excludes internal logic and data movement; State Diagrams excludes system-wide data flow and multi-actor permissions) — forcing all three questions into one diagram would either produce an unreadably overloaded diagram or silently drop some of the requested information because that diagram type was never designed to represent it.